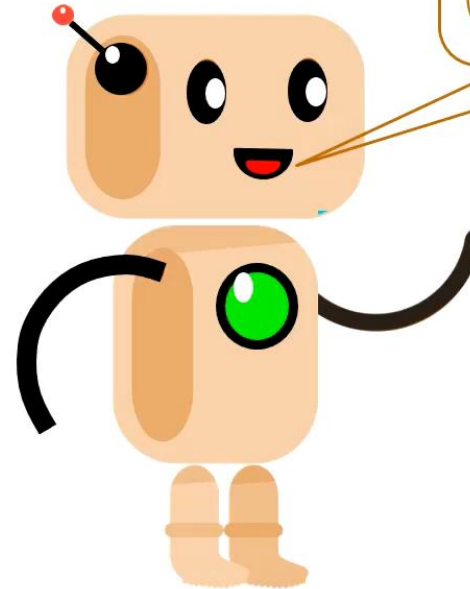
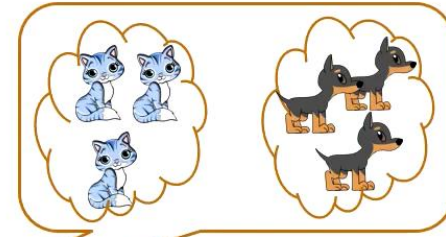
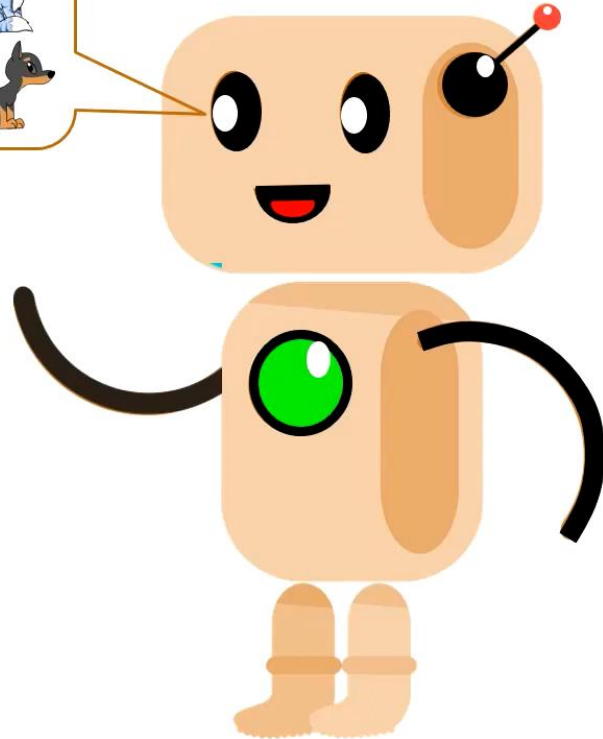
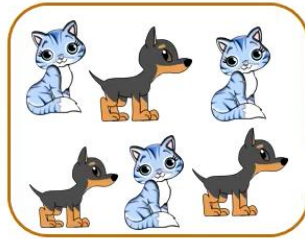


Unit 3

- Unsupervised learning: Clustering - K-means, DBSCAN, Gaussian Mixture Model, Parameter Estimation: MLE and Bayesian Estimate, Expectation Maximization, Case study on advanced unsupervised ML techniques for regression and classification. Artificial Neural Networks: Multi-layer Perceptron, Back Propagation Algorithm, ANN applications to classification and regression

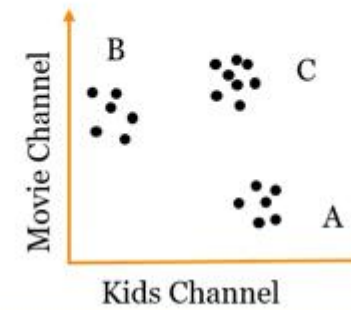
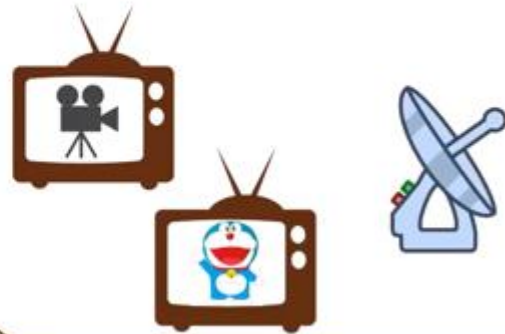


Unsupervised Learning



Clustering

Set of objects are divided into clusters. A cluster has objects with same properties.



Supervised Learning

vs

Unsupervised Learning

Training data is **labeled**

Training
Data

Training data is **unlabeled**

It takes **feedback** to verify whether the predicted output is correct or not.

Output

It identifies **hidden pattern** into the input data to predict the output.

Both **input** and its **output** are provided to the algorithms.

Training

Only the **input** data is provided to the algorithms.

Classification and Regression

Categorize

Clustering and Association



K Means Clustering

- “**Clustering** is the task of dividing the population or **data points into several groups** such that data points in the same groups are more similar to other data points in the same group than those in other groups.”
- K-Means clustering is an unsupervised learning algorithm.
- There is no labelled data for this clustering, unlike in supervised learning. K-Means performs the division of objects into clusters that share similarities and are dissimilar to the objects belonging to another cluster.
- The term ‘K’ is a number. We need to tell the system how many clusters you need to create.
- For example, $K = 2$ refers to two clusters.
- There is a way of finding out the best or optimum value of K for a given data.



Assume the data points has to be grouped into 3 clusters where the points are

A1 (2, 10), A2 (2, 5), A3 (8, 4),

B1 (5, 8), B2 (7, 5) B3 (6, 4),

C1 (1, 2), C2 (4, 9).

Perform for 2 iterations.

Solution:

First choose the centers of the clusters .Assign A1, B1 and C1 as the center of the cluster respectively.



Data Points			Distance to						Cluster	New Cluster
			2	10	5	8	1	2		
A1	2	10								
A2	2	5								
A3	8	4								
B1	5	8								
B2	7	5								
B3	6	4								
C1	1	2								
C2	4	9								

$$d(p_1, p_2) = \sqrt{(x_2 - x_1)^2 + (y_2 - y_1)^2}$$



1st Iteration

BI

Data Points			AI		Distance to		CI		Cluster	New Cluster
			2	10	5	8	1	2		
A1	2	10	0.00		3.61		8.06		1	
A2	2	5	5.00		4.24		3.16		3	
A3	8	4	8.49		5.00		7.28		2	
B1	5	8	3.61		0.00		7.21		2	
B2	7	5	7.07		3.61		6.71		2	
B3	6	4	7.21		4.12		5.39		2	
C1	1	2	8.06		7.21		0.00		3	
C2	4	9	2.24		1.41		7.62		2	



Compute New Centroids

Cluster 1

Points: **A1 (2,10)**

C1 = (2, 10)

A2 (2,5)

C1 (1,2)

Cluster 2

A3 (8,4)

B1 (5,8)

B2 (7,5)

B3 (6,4)

C2 (4,9)

Mean x

$$(2 + 1)/2 = 1.5$$

Mean y

$$(5 + 2)/2 = 3.5$$

New centroid

(1.5,3.5)

Mean x:

$$(8 + 5 + 7 + 6 + 4)/5 = 30/5 = 6$$

Mean y:

$$(4 + 8 + 5 + 4 + 9)/5 = 30/5 = 6$$

New centroid:

(6,6)

Current Centroids:

A1: (2, 10)

B1: (6, 6)

C1: (1.5, 3.5)



Data Points			Distance to						Cluster	New Cluster
			2	10	6	6	1.5	3.5		
A1	2	10	0.00		5.66		6.52		1	1
A2	2	5	5.00		4.12		1.58		3	3
A3	8	4	8.49		2.83		6.52		2	2
B1	5	8	3.61		2.24		5.70		2	2
B2	7	5	7.07		1.41		5.70		2	2
B3	6	4	7.21		2.00		4.53		2	2
C1	1	2	8.06		6.40		1.58		3	3
C2	4	9	2.24		3.61		6.04		2	1

New Centroids→

Current Centroids:

A1: (3, 9.5)

B1: (6.5, 5.25)

C1: (1.5, 3.5)



Data Points			Distance to						Cluster	New Cluster
			3	9.5	6.5	5.25	1.5	3.5		
A1	2	10	1.12		6.54		6.52		1	1
A2	2	5	4.61		4.51		1.58		3	3
A3	8	4	7.43		1.95		6.52		2	2
B1	5	8	2.50		3.13		5.70		2	1
B2	7	5	6.02		0.56		5.70		2	2
B3	6	4	6.26		1.35		4.53		2	2
C1	1	2	7.76		6.39		1.58		3	3
C2	4	9	1.12		4.51		6.04		1	1



Disadvantage of K-Means

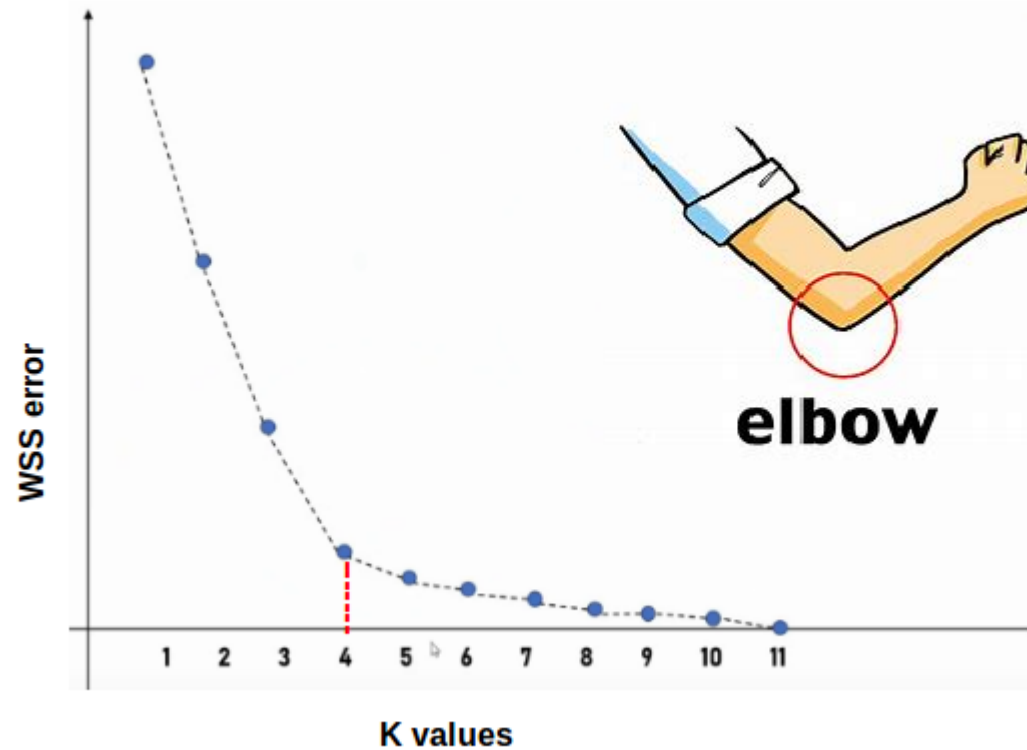
- Sensitivity to Outliers
- Inaccurate clustering due to the choice of initial centroids.



Optimal method of finding number of clusters

Elbow method

calculate WCSS
(Within Cluster
Sum of Squares)



$$WCSS = \sum_{i=1}^k \sum_{x \in C_i} (x - \mu_i)^2$$

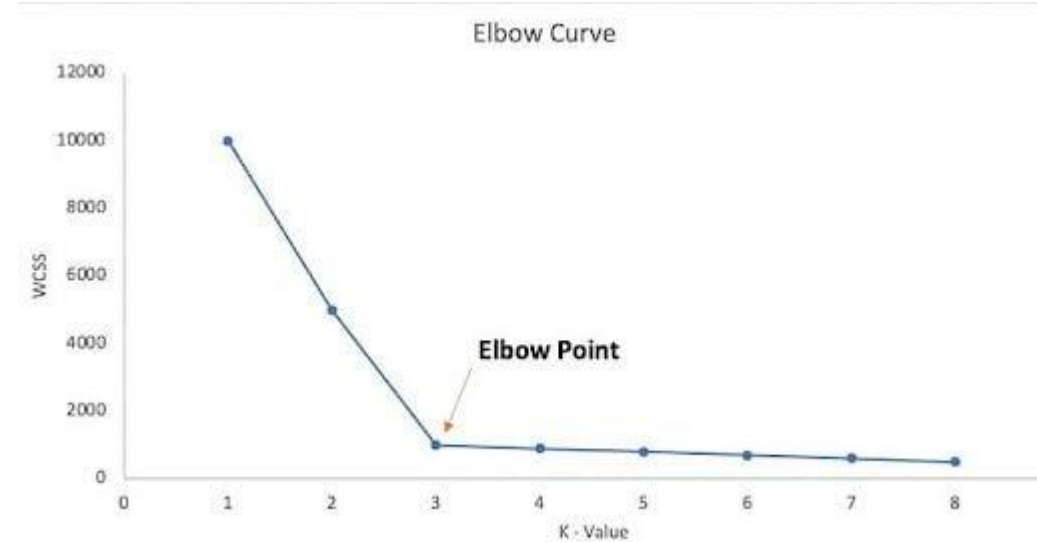
$k \rightarrow$ number of clusters

$C_i \rightarrow$ set of points in cluster i

$x \rightarrow$ data point in the cluster

$\mu_i \rightarrow$ centroid (mean) of cluster

$(x - \mu_i)^2 \rightarrow$ squared distance between point and centroid



$$SSE = \sum (x - x_c)^2 + (y - y_c)^2$$

where (x_c, y_c) is the centroid.

Point	X	Y	$(x-x_c)^2$	$(y-y_c)^2$	SSE
A1	2	10	5.64	17.02	22.66
A2	2	5	5.64	0.77	6.41
A3	8	4	13.14	3.52	16.66
B1	5	8	0.39	4.51	4.90
B2	7	5	6.89	0.77	7.66
B3	6	4	2.64	3.52	6.16
C1	1	2	11.39	15.01	26.40
C2	4	9	0.14	9.77	9.91

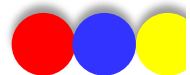
K = 1

$$x_c = \frac{2 + 2 + 8 + 5 + 7 + 6 + 1 + 4}{8} = 4.375$$

Centroid = **(4.375 , 5.875)**

$$y_c = \frac{10 + 5 + 4 + 8 + 5 + 4 + 2 + 9}{8} = 5.875$$

SSE1=100.76



K	SSE
1	100.76
2	51.47
3	13.43



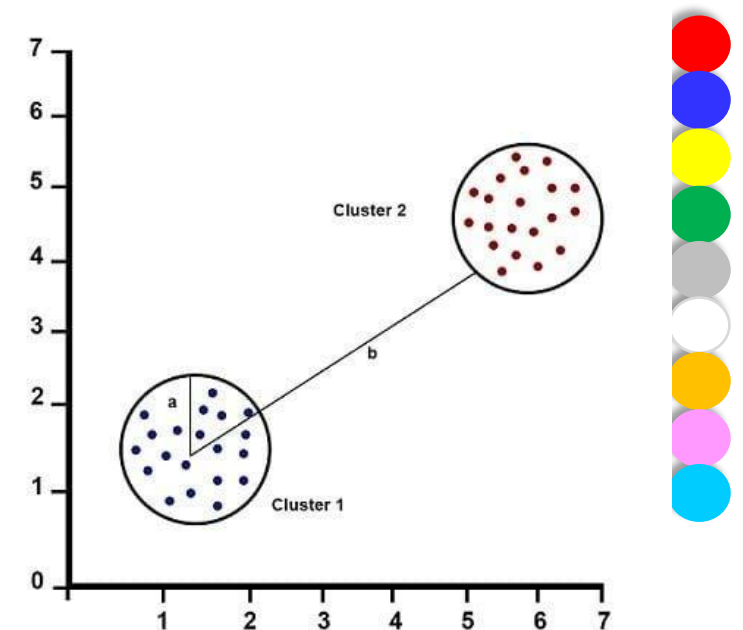
Silhouette Score

- The Silhouette score is a very useful method to find the number of K when the elbow method doesn't show a clear elbow point. The **Silhouette Score** is a clustering evaluation metric that measures how similar a data point is to its own cluster compared to other clusters, with values ranging from **-1 to 1**.
- The value of the Silhouette score ranges from **-1 to 1**. Following is the interpretation of the Silhouette score.
- **1**: Points are perfectly assigned in a cluster and clusters are easily distinguishable.
- **0**: Clusters are overlapping.
- **-1**: Points are wrongly assigned in a cluster.

$$\text{Silhouette Score} = (b-a) / \max(a,b)$$

a = average intra-cluster distance
(i.e. the average distance between each point within a cluster).

b = average inter-cluster distance
(i.e. the average distance between all clusters).



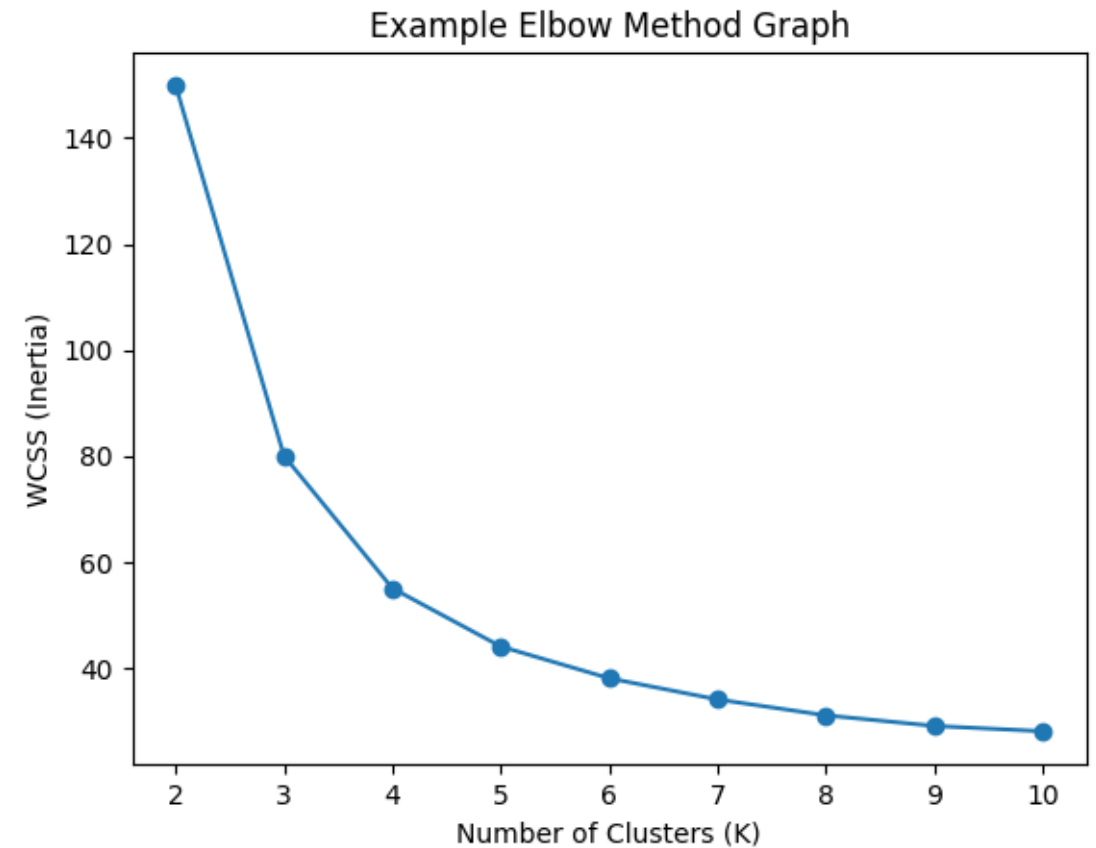
```
#install yellowbrick to vizualize the Elbow curve
!pip install yellowbrick

from sklearn import datasets
from sklearn.cluster import KMeans
from yellowbrick.cluster import KElbowVisualizer

# Load the IRIS dataset
iris = datasets.load_iris()
X = iris.data
y = iris.target

# Instantiate the clustering model and visualizer
km = KMeans(random_state=42)
visualizer = KElbowVisualizer(km, k=(2,10))

visualizer.fit(X)      # Fit the data to the visualizer
visualizer.show()      # Finalize and render the figure
```



```

from sklearn.datasets import load_iris
from sklearn.cluster import KMeans
from sklearn.metrics import silhouette_score
import matplotlib.pyplot as plt

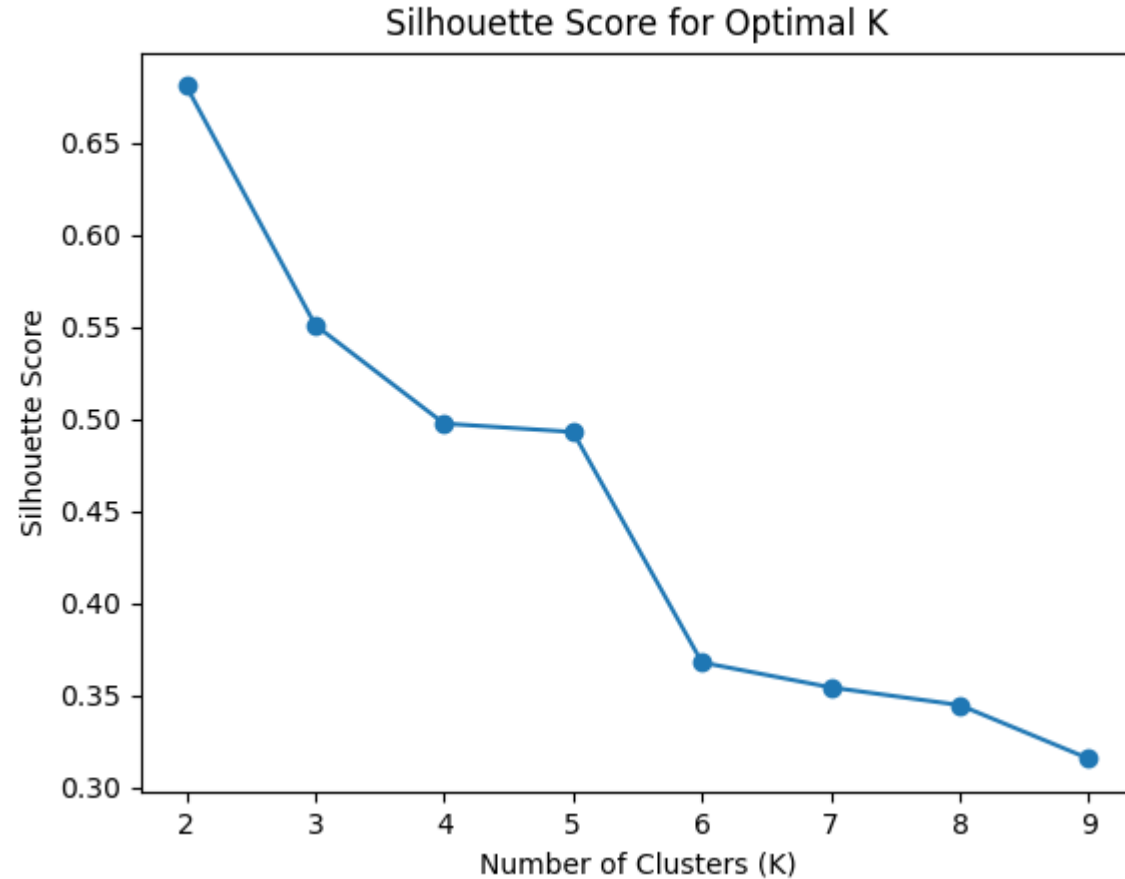
# Load dataset
iris = load_iris()
X = iris.data

scores = []
K = range(2,10)

# Compute silhouette score for different K values
for k in K:
    kmeans = KMeans(n_clusters=k, random_state=42)
    labels = kmeans.fit_predict(X)
    score = silhouette_score(X, labels)
    scores.append(score)

# Plot the graph
plt.plot(K, scores, marker='o')
plt.xlabel("Number of Clusters (K)")
plt.ylabel("Silhouette Score")
plt.title("Silhouette Score for Optimal K")
plt.show()

```



Problem 2

- Apply the **K-Means clustering algorithm** to the dataset **P1(1,1)**, **P2(1.5,2)**, **P3(3,4)**, **P4(5,7)**, **P5(3.5,5)**, and **P6(4.5,5)** with the number of clusters **K = 2**. The **initial centroids** are **$C_1 = (1,1)$** and **$C_2 = (5,7)$** . Using the **Euclidean distance formula**, compute the distance of each point from both centroids and assign the points to the nearest cluster. Calculate the **new centroids** after cluster assignment (**Iteration 1**) and repeat the process once more to perform **Iteration 2**. Show the **distance calculations, cluster assignments, and updated centroids** for both iterations.

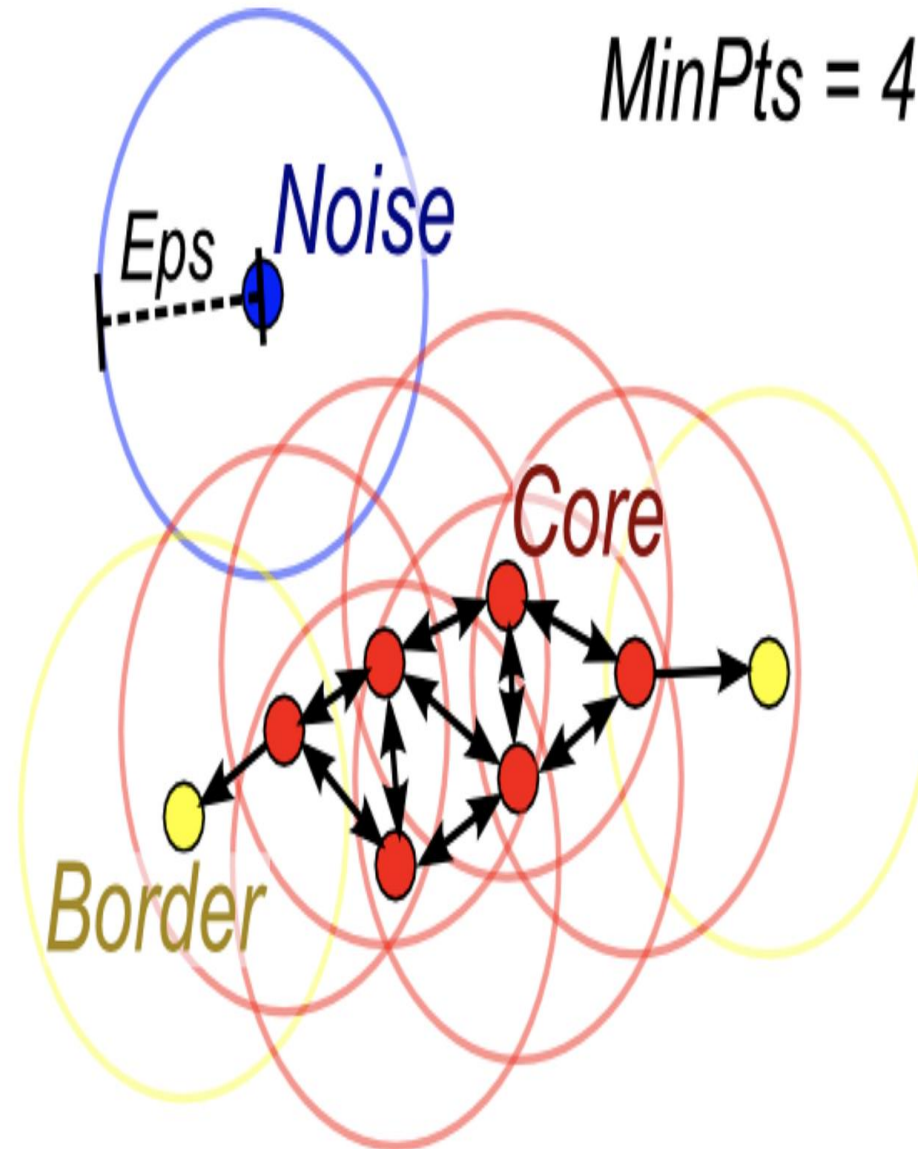


DBSCAN



- **DBSCAN** (**D**ensity-**B**ased **S**patial **C**lustering of **A**pplications with **N**oise)
- **DBSCAN** is an **unsupervised clustering algorithm** that groups data points based on **density**. It forms clusters where **points are closely packed together** and identifies **points in low-density regions as noise (outliers)**.

$MinPts = 4$



Red: Core Points

Yellow: Border points. Still part of the cluster because it's within epsilon of a core point, but not does not meet the min_points criteria

Blue: Noise point. Not assigned to a cluster



Key Parameters in DBSCAN

1. eps: This defines the radius of the neighborhood around a data point. If the distance between two points is less than or equal to eps, they are considered neighbors. A common method to determine eps is by analyzing the k-distance graph. Choosing the right eps is important

- If eps is too small most points will be classified as noise.
- If eps is too large clusters may merge and the algorithm may fail to distinguish between them.

2. MinPts: This is the minimum number of points required within the **eps** radius to form a dense region. *For most cases a minimum value of **MinPts = 3** is recommended.*

DBSCAN works by categorizing data points into three types:

- Core points which have a sufficient number of neighbors within a specified radius (epsilon)
- Border points which are near core points but lack enough neighbors to be core points themselves
- Noise points which do not belong to any cluster.



- Apply the DBSCAN algorithm to the given data points and
- Create the clusters with
- $\text{minPts} = 4$ and
- $\text{epsilon} (\epsilon) = 1.9$.

Data Points:

P1: (3, 7)

P2: (4, 6)

P3: (5, 5)

P4: (6, 4)

P5: (7, 3)

P6: (6, 2)

P7: (7, 2)

P8: (8, 4)

P9: (3, 3)

P10: (2, 6)

P11: (3, 5)

P12: (2, 4)



- Use Eucladian distance and calculate the distance between each points.

$$\textit{Distance}(A(x_1, y_1), B(x_2, y_2)) = \sqrt{(x_2 - x_1)^2 + (y_2 - y_1)^2}$$



	P1 (3,7)	P2 (4,6)	P3 (5,5)	P4 (6,4)	P5 (7,3)	P6 (6,2)	P7 (7,2)	P8 (8,4)	P9 (3,3)	P10 (2,6)	P11 (3,5)	P12 (2,4)
P1 (3,7)	0											
P2 (4,6)	1.41	0										
P3 (5,5)	2.83	1.41	0									
P4 (6,4)	4.24	2.83	1.41	0								
P5 (7,3)	5.66	4.24	2.83	1.41	0							
P6 (6,2)	5.83	4.47	3.16	2.00	1.41	0						
P7 (7,2)	6.40	5.00	3.61	2.24	1.00	1.00	0					
P8 (8,4)	5.83	4.47	3.16	2.00	1.41	2.83	2.24	0				
P9 (3,3)	4.00	3.16	2.83	3.16	4.00	3.16	4.12	5.10	0			
P10 (2,6)	1.41	2.00	3.16	4.47	5.83	5.66	6.40	6.32	3.16	0		
P11 (3,5)	2.00	1.41	2.00	3.16	4.47	4.24	5.00	5.10	2.00	1.41	0	
P12 (2,4)	3.16	2.83	3.16	4.00	5.10	4.47	5.39	6.00	1.41	2.00	1.41	0

	P1	P2	P3	P4	P5	P6	P7	P8	P9	P10	P11	P12	Point	ε-Neighbours
P1	0												P1	P2, P10
P2	1.41	0											P2	P1, P3, P11
P3	2.83	1.41	0										P3	P2, P4
P4	4.24	2.83	1.41	0									P4	P3, P5
P5	5.66	4.24	2.83	1.41	0								P5	P4, P6, P7, P8
P6	5.83	4.47	3.16	2.00	1.41	0							P6	P5, P7
P7	6.40	5.00	3.61	2.24	1.00	1.00	0						P7	P6, P5
P8	5.83	4.47	3.16	2.00	1.41	2.83	2.24	0					P8	P5
P9	4.00	3.16	2.83	3.16	4.00	3.16	4.12	5.10	0				P9	P12
P10	1.41	2.00	3.16	4.47	5.83	5.66	6.40	6.32	3.16	0			P10	P1, P11
P11	2.00	1.41	2.00	3.16	4.47	4.24	5.00	5.10	2.00	1.41	0		P11	P2, P10, P12
P12	3.16	2.83	3.16	4.00	5.10	4.47	5.39	6.00	1.41	2.00	1.41	0	P12	P9, P11

- minPts = 4 and
- epsilon (ε) = 1.9.



ARITA
A VIDYAPEETHAM | School of Engineering
BENGALURU | CHENNAI | COIMBATORE

- minPts = 4 and
- epsilon (ϵ) = 1.9.

Point	ϵ -Neighbours
P1	P2, P10
P2	P1, P3, P11
P3	P2, P4
P4	P3, P5
P5	P4, P6, P7, P8
P6	P5, P7
P7	P5, P6
P8	P5
P9	P12
P10	P1, P11
P11	P2, P10, P12
P12	P9, P11

Point	Noise	Border
P1	Noise	Border
P2	Core	
P3	Noise	Border
P4	Noise	Border
P5	Core	
P6	Noise	Border
P7	Noise	Border
P8	Noise	Border
P9	Noise	
P10	Noise	Border
P11	Core	
P12	Noise	Border

P1 (should be in any core point then it is border)

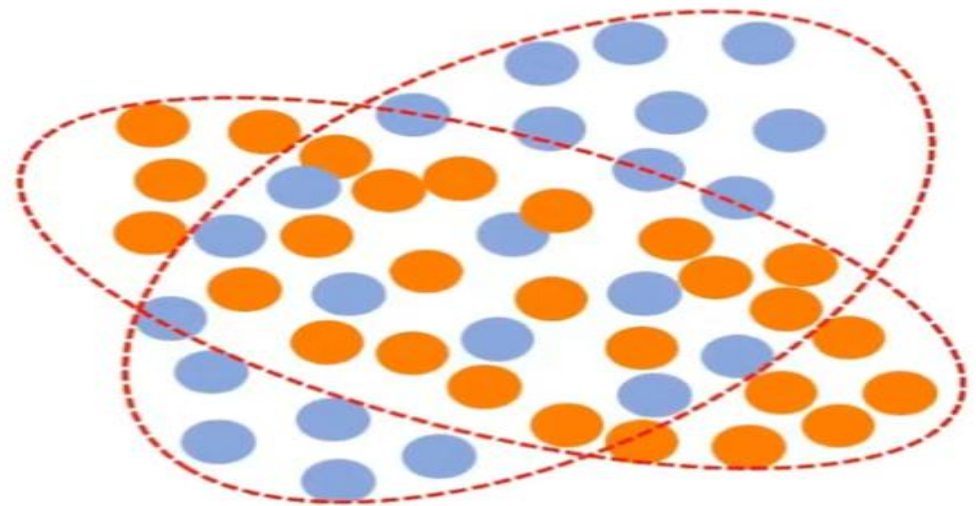
Noise

Border

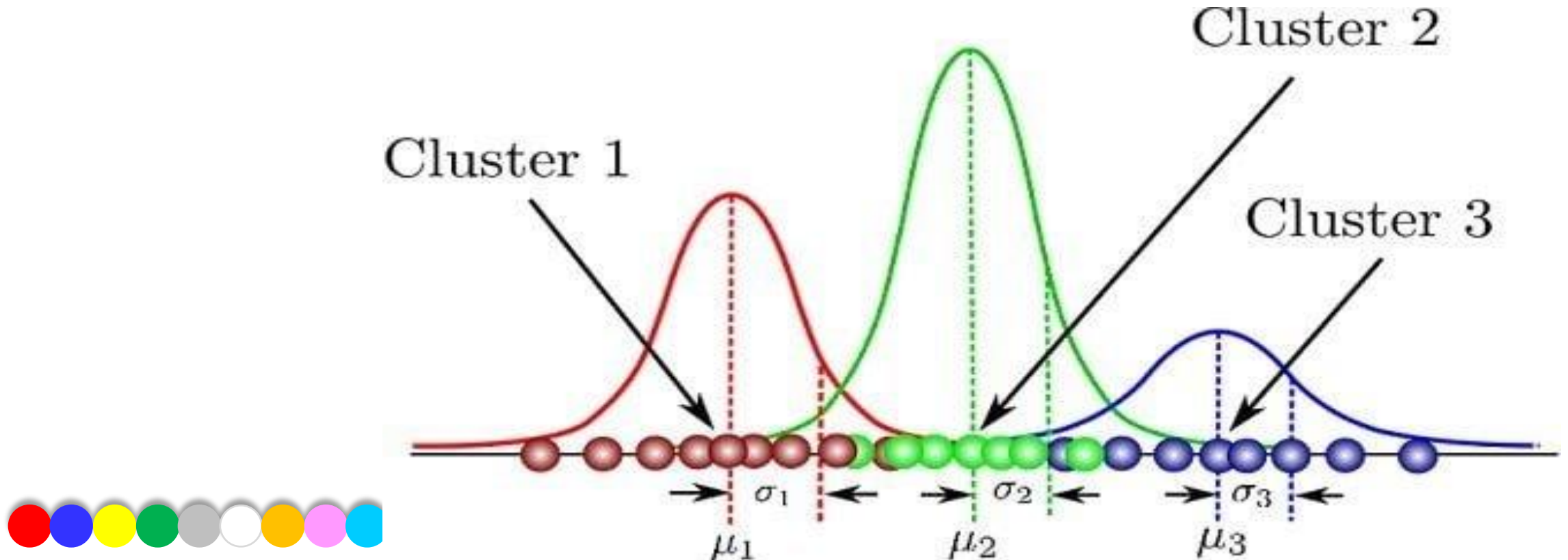


Gaussian Mixture Model

- A Gaussian mixture model (GMM) is a machine learning method used to determine the probability each data point belongs to a given cluster.
- The model is a soft clustering method used in unsupervised learning.
- **Gaussian Mixture Model Defined**
- A Gaussian mixture model is a soft clustering technique used in unsupervised learning to determine the probability that a given data point belongs to a cluster.
- It's composed of several Gaussians, each identified by $k \in \{1, \dots, K\}$, where K is the number of clusters in a data set.



- Each Gaussian k in the mixture is comprised of the following parameters:
- A mean μ that defines its center.
- A covariance Σ that defines its width. This would be equivalent to the dimensions of an ellipsoid in a multivariate scenario.
- A mixing probability π that defines how big or small the Gaussian function will be.



Real-World Example: Customer Segmentation in a Supermarket

- Imagine a supermarket collects customer shopping data, such as:
 - Total amount spent per month
 - Frequency of visits per month
 - Average bill amount per visit

Now, the store wants to categorize customers into 3 types:

- Budget Shoppers (low spending, low frequency)
 - Regular Shoppers (medium spending, medium frequency)
 - Premium Shoppers (high spending, high frequency)
-
- Each of these customer categories can be represented as a Gaussian distribution. However, a customer may not always belong entirely to one category. For example:
 - A customer may have 60% probability of being a Regular Shopper and 40% probability of being a Premium Shopper.
 - Another may have 90% probability of being a Budget Shopper.
 - This is where GMM is useful – it allows a probabilistic assignment of customers to clusters.



Multi-layer Perceptron, Back Propagation Algorithm, ANN applications to classification and regression

